

Arena Defender - A 2D Survival Game with MonoGame

Project Overview

Throughout this module, you have learned the foundations of C# programming, object-oriented programming, software design principles, mathematics for game development, and an introduction to MonoGame.

This project gives you the opportunity to bring all of these concepts together by developing a complete 2D game from scratch using **MonoGame**.

Unlike previous assignments where you analyzed existing games, this project requires you to think and work as a game developer. You will design and implement a playable game while demonstrating your understanding of programming, software engineering, and mathematical concepts covered throughout the module.

Project Theme

Arena Defender

Develop a **2D survival game** in which the player must survive waves of enemies for as long as possible while earning points by defeating enemies and collecting power-ups.

The game should present an increasing level of difficulty over time through enemy spawning, movement, or other gameplay mechanics.

Learning Outcomes

By completing this project, you should be able to:

- Develop a complete game using C# and MonoGame.
 - Apply object-oriented programming and SOLID principles in a well-structured codebase.
 - Implement mathematical concepts to solve gameplay problems.
 - Build reusable and maintainable software components.
 - Demonstrate effective software documentation and communication skills.
-

Project Requirements

1. You are required to use **Visual Studio Code** for this project. This is especially important for students using **macOS**, as Visual Studio Code provides the recommended and most consistent development environment for MonoGame across platforms.
2. Please ensure that you have **.NET 9.0 SDK** or later installed before starting this project. MonoGame requires a minimum of **.NET 9.0 SDK** to build and run successfully. If your current installation is an earlier version, upgrade it before creating your project.

Your game must include all of the following features:

1. Player System

Implement a controllable player character with the following features:

- Keyboard-controlled movement (WASD or Arrow Keys)
 - Player health
 - Score tracking
 - Lives (optional if health reaches zero and game ends)
 - Player sprite or placeholder graphics
-

2. Enemy System

Implement a system that continuously spawns enemies.

Your game must include **at least two different enemy types**, each with different behaviours or characteristics.

Examples include:

- Fast but weak enemy
- Slow but strong enemy
- Standard enemy

You are free to design additional enemy types.

3. Projectile / Attack System

The player must be able to defend themselves.

Examples include:

- Shooting projectiles
- Energy blasts
- Magic attacks
- Any other reasonable attack mechanic

Projectiles should interact correctly with enemies. A combination of these attack features can be implemented for the player with different inputs or trigger states.

4. Collision System

Implement collision detection for at least the following:

- Player and enemies
- Projectiles and enemies
- Player and collectibles or power-ups

Collisions should produce appropriate gameplay outcomes such as taking damage, destroying enemies, collecting items, or increasing score.

Creative freedom: It is up to you to decide what happens when the player comes in contact with the pick-up objects or shoot at the pick-up object.

5. Power-Up System

Implement **at least one** different power-ups.

Examples include:

- Extra health
- Increased movement speed
- Temporary shield
- Increased attack power
- Faster shooting

You may design additional power-ups.

6. Scoring System

Implement a scoring system that rewards players for gameplay.

Examples include:

- Defeating enemies
- Collecting items
- Surviving longer
- Completing objectives

Display the player's score during gameplay.

7. User Interface

Your game should display appropriate gameplay information, including:

- Start UI with the game name and a "Start" or "Play" button at least.
- Health
- Score
- Remaining lives (if applicable)
- Game Over or Victory screen

You are encouraged to make the interface clean, easy to understand and navigate.

8. Mathematics Requirements

Your implementation must demonstrate practical use of the mathematical concepts covered during the module.

At a minimum, your game should demonstrate:

- **Distance:** Examples include, enemy detection radius, collectible pickup range, interaction distance
- **Direction and Vectors:** Examples include, player movement, enemy movement, projectile movement
- **Algebra:** Examples include, health calculations, damage calculations, scoring, difficulty scaling, power-up effects
- **Dot Product:** Use the dot product in at least one gameplay mechanic. Possible examples include, enemy field of view, target detection, facing direction checks, aim assistance.
- **Cross Product:** Use the cross product in at least one gameplay mechanic. Possible examples include, determining left or right turns, character orientation, rotational behaviour

- **Linear Interpolation (Lerp):** Use linear interpolation in at least **three** different situations. Examples include, smooth player movement, camera movement, health bar animation, colour transitions, UI animations, fade effects
-

Software Design

Your solution should demonstrate good software engineering practices by applying the C# programming concepts, object-oriented programming principles, and SOLID principles covered throughout this module. Your code should be modular, reusable, well-organized, and appropriately documented.

Unit Testing

To demonstrate your understanding of software quality and test-driven development principles introduced in this module, your project must include automated unit tests.

Your solution must include:

- A separate test project using **NUnit** or **xUnit**.
- **At least ten (10) meaningful unit tests** that verify the correctness of important game logic.
- Tests should focus on business logic and mathematical calculations rather than graphics or rendering.

Examples of components that can be tested include:

- Health and damage calculations
- Score calculations
- Enemy spawning logic
- Distance calculations
- Dot product or vector calculations
- Collision-related helper methods
- Utility classes
- Power-up effects
- Player statistics
- Difficulty scaling

Your tests should be well named, independent, repeatable, and demonstrate that your code has been designed to be testable.

Deliverables/Submission Requirements

1. GitHub Repository:

- Upload your complete project to a **public GitHub repository**.
- Include a `README.md` file with:
 - Description of the game
 - How to play
 - How to run the project
- XML documentation embedded in code

2. Application Design Document:

- A document describing the project's **architecture and idea**.
- Justify why you chose certain data structures or programming patterns.

3. Testing Strategy:

- Describe your **testing approach** (including unit tests, edge cases, and test results).
- `UnitTests` folder with all test cases

4. Video Demonstration:

- A **recording of the detailed gameplay and code walkthrough** (must be accessible, 10 - 15 minutes).

5. Submission Guidelines/Summary:

- Submit a document with links to:
 - GitHub repository (include code and test folder)
 - Architecture document
 - Testing strategy document
 - Video demo
- A **presentation date** will be scheduled after submission.

6. Possible Presentation Questions:

- How does your UI communicate with your game logic?
- Did you separate game logic from UI logic? If yes, how?
- How did you apply **encapsulation** in your classes?
- Did you use **inheritance or interfaces** anywhere in your project?
- Demonstrate the use of **Abstraction** in your project
- Why did you choose **the Data Structure(s)** that you used instead of another structure?
- Where did you use an **Enum**, and what problem did it solve?
- Which functions did you test in your unit tests?
- Why did you choose those specific tests?
- Can you show an example of a failing test and how you fixed it?
- How could you implement **leaderboards**?

Arena Defender - A 2D Survival Game with MonoGame					
Criteria	Ratings				Pts
Gameplay Functionality	5 to >3.0 pts Exemplary All required gameplay systems are fully implemented, polished, and function correctly.	3 to >1.0 pts Proficient Most required systems are implemented with minor issues.	1 to >0.0 pts Developing Core gameplay is functional but several required features are incomplete.	0 pts Beginning Significant gameplay features are missing or non-functional.	5 pts
Application of C# Programming Concepts	5 to >3.0 pts Exemplary Demonstrates excellent use of C# fundamentals, clean coding practices, modular design, and appropriate use of object-oriented programming and SOLID principles.	3 to >1.0 pts Proficient Good implementation with minor structural issues.	1 to >0.0 pts Developing Basic implementation with limited application of best practices.	0 pts Beginning Poor code organization or limited evidence of concepts taught.	5 pts
Application of Mathematics in Game Development	8 to >4.0 pts Exemplary Effectively integrates all required mathematical concepts with clear evidence of their practical use in gameplay.	4 to >2.0 pts Proficient Most mathematical concepts are implemented correctly.	2 to >0.0 pts Developing Some concepts are implemented but with limited effectiveness or explanation.	0 pts Beginning Limited or incorrect implementation of required mathematical concepts.	8 pts

Unit Testing	5 to >3.0 pts Exemplary Includes a well-structured test project with at least ten meaningful unit tests using NUnit or xUnit. All tests pass successfully, cover important game logic, and demonstrate good testing practices.	3 to >1.0 pts Proficient Includes at least ten tests with minor issues in coverage or quality.	1 to >0.0 pts Developing Includes fewer than ten tests or tests provide limited coverage of important functionality.	0 pts Beginning Unit tests are missing, do not execute successfully, or provide little evidence of testing.	5 pts
Exception Handling	3 to >2.0 pts Exemplary Robust input and runtime error handling	2 to >1.0 pts Proficient Basic try/catch logic included	1 to >0.0 pts Developing Minimal checks in place	0 pts Beginning No error handling present	3 pts
Software Design and Documentation	2 to >1.5 pts Exemplary Report is comprehensive, professionally presented, clearly explains system architecture, mathematical implementation, testing approach, and development process, with appropriate diagrams.	1.5 to >1.0 pts Proficient Good documentation with minor omissions.	1 to >0.0 pts Developing Adequate documentation but lacking depth or clarity.	0 pts Beginning Documentation is incomplete, poorly organized, or missing key sections.	2 pts
Professional Presentation and Submission	2 to >1.5 pts Exemplary All required deliverables are submitted, the gameplay demonstration is clear and professional, the GitHub repository is well organized and accessible, and all required files are included.	1.5 to >1.0 pts Proficient Most submission requirements are met with minor issues.	1 to >0.0 pts Developing Submission is complete but lacks professionalism or misses some components.	0 pts Beginning Missing required deliverables or inaccessible repository.	2 pts
Total Points: 30					